



**Desarrollo de aplicaciones multiplataforma**

## **MVP**

### **GestNutri**

**Realizado por :**

Jilder Alex Dionisio Rojas

Genny D. Navarrete Aguilar

Diana Gpe. Bojorquez Cetina

Luisa Fernanda Ortiz Baeza

**Docente:**

**Ing. Carlos Canto**

## **Resumen**

Este plan establece cómo se organizará, desarrollará, integrará y controlará GestNutri, una aplicación web para la gestión y seguimiento de pacientes de nutrición. El equipo estará formado por cuatro integrantes que trabajarán desde computadoras diferentes sobre un único repositorio de GitHub. El proyecto utilizará Scrum para organizar el trabajo en Sprints y tendrá como fecha límite el 10 de octubre de 2026. La solución se desarrollará con React, Django, Django REST Framework y PostgreSQL.

La inteligencia artificial se utilizará en dos niveles claramente diferenciados. Primero, cada integrante podrá utilizar IA en su propia computadora como apoyo para analizar, programar, probar, documentar y revisar su tarea. Segundo, existirá un Agente Orquestador con contexto global del proyecto, encargado de relacionar el trabajo publicado por los integrantes, detectar dependencias e inconsistencias y sugerir acciones de integración. El Agente Orquestador no sustituye GitHub, Scrum ni la revisión humana.

## Índice

|                                                 |    |
|-------------------------------------------------|----|
| Resumen                                         | 2  |
| 1. Descripción, objetivos y alcance             | 4  |
| 2. Supuestos y restricciones                    | 4  |
| 3. Organización del Scrum Team                  | 5  |
| 4. Forma de trabajo con Scrum                   | 6  |
| 5. Product Backlog inicial y trazabilidad       | 6  |
| 6. Plan de Sprints hasta el 10 de octubre       | 7  |
| 7. Arquitectura y tecnologías                   | 8  |
| 8. Desarrollo distribuido con Git y GitHub      | 8  |
| 9. Forma de trabajo con inteligencia artificial | 9  |
| 10. Agente Orquestador con contexto global      | 10 |
| 11. Gestión de requisitos y cambios             | 11 |
| 12. Gestión de calidad, pruebas y CI            | 11 |
| 13. Seguridad, riesgos y comunicación           | 12 |
| 14. Seguimiento y métricas                      | 13 |
| 15. Entregables y criterios de aceptación       | 13 |
| 16. Conclusiones                                | 14 |
| Referencias                                     | 14 |

## Plan de gestión de proyectos de software

### 1. Descripción, objetivos y alcance

GestNutri es una aplicación web orientada a profesionales de nutrición. Su propósito es centralizar la información de los pacientes y facilitar el seguimiento entre consultas. El proyecto responde a la necesidad de reducir la dispersión de información en notas, archivos y herramientas independientes, y de disponer de una visión clara de la evolución de cada paciente.

El objetivo general es desarrollar, integrar y validar antes del 10 de octubre de 2026 un producto mínimo viable (MVP) que permita administrar pacientes, registrar consultas y mediciones, consultar historial, comparar automáticamente la evolución y utilizar inteligencia artificial de forma controlada para apoyar la redacción de notas.

El Product Goal es disponer, al cierre del proyecto, de una versión demostrable de GestNutri que permita al nutricionista completar el flujo básico de seguimiento de un paciente desde su registro hasta la revisión de su evolución, manteniendo trazabilidad del desarrollo, seguridad básica y revisión humana de las funciones asistidas por IA.

| Incluido en el MVP                                                                                                                                                                                                                                                               | Fuera del MVP                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autenticación; lista, búsqueda, alta y edición de pacientes; expediente; contacto y WhatsApp; consultas; mediciones; historial; comparación de progreso; notas, alergias y restricciones; documentos esenciales; resumen de seguimiento; formalización opcional de notas con IA. | Catálogo y comparador de productos; reconocimiento de etiquetas por fotografía; recomendaciones comerciales; alianzas o patrocinios; y funciones que no sean indispensables para validar el flujo principal. |

### 2. Supuestos y restricciones

| Elemento     | Definición                                                   |
|--------------|--------------------------------------------------------------|
| Fecha límite | 10 de octubre de 2026.                                       |
| Equipo       | Cuatro integrantes trabajando desde computadoras diferentes. |

|                     |                                                                                    |
|---------------------|------------------------------------------------------------------------------------|
| Producto            | Un solo proyecto GestNutri y un único repositorio central en GitHub.               |
| Marco de trabajo    | Scrum con tres Sprints cortos por el tiempo disponible.                            |
| Tecnologías         | React, Django, Django REST Framework y PostgreSQL.                                 |
| IA                  | IA de apoyo en cada computadora y un Agente Orquestador con contexto global.       |
| Datos de desarrollo | Datos ficticios o anonimizados; no se utilizarán datos reales en prompts externos. |
| Prioridad           | Proteger las funciones esenciales del MVP ante cualquier desviación de tiempo.     |

### 3. Organización del Scrum Team

El Scrum Team estará formado por un Product Owner, un Scrum Master y los Developers. Los cuatro integrantes participan como Developers en la creación del Increment; las responsabilidades Scrum coexistirán con las especialidades técnicas y no generarán subequipos independientes (Schwaber & Sutherland, 2020).

| Responsabilidad | Asignación                                              | Función                                                                                                                                      |
|-----------------|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Product Owner   | Por definir por el equipo antes del Sprint 1 / Rotativo | Ordenar el Product Backlog, aclarar necesidades y criterios de aceptación, mantener el Product Goal y priorizar valor.                       |
| Scrum Master    | Por definir por el equipo antes del Sprint 1 / Rotativo | Facilitar Scrum, ayudar a eliminar impedimentos, cuidar los eventos y promover mejora continua, trabaja de la mano con el agente orquestador |
| Frontend        | Jilder Alex Dionisio Rojas                              | React, interfaz e integración con la API.                                                                                                    |
| Backend         | Genny D. Navarrete Aguilar                              | Django, Django REST Framework, endpoints, permisos y lógica de negocio.                                                                      |

|                 |                             |                                                                         |
|-----------------|-----------------------------|-------------------------------------------------------------------------|
| Datos / Backend | Diana Gpe. Bojorquez Cetina | PostgreSQL, modelos, migraciones y consistencia de datos.               |
| QA / CI         | Luisa Fernanda Ortiz Baeza  | Pruebas, GitHub Actions, calidad y apoyo técnico al Agente Orquestador. |

La responsabilidad final sobre el Increment es colectiva. Una especialidad indica quién lidera técnicamente una actividad, pero no significa que esa persona sea la única autorizada para trabajar en ella.

#### 4. Forma de trabajo con Scrum

Cada Sprint tendrá un Sprint Goal y deberá producir un Increment integrado y demostrable. En Sprint Planning se seleccionarán las historias prioritarias y se identificarán dependencias; la Daily Scrum, de hasta 15 minutos en los días de trabajo, permitirá inspeccionar el avance y ajustar el plan; la Sprint Review demostrará el resultado; y la Sprint Retrospective identificará mejoras para el siguiente ciclo.

| Artefacto / compromiso         | Uso en GestNutri                                                                                    |
|--------------------------------|-----------------------------------------------------------------------------------------------------|
| Product Backlog / Product Goal | Concentra las historias del producto y mantiene visible el objetivo general del MVP.                |
| Sprint Backlog / Sprint Goal   | Contiene las historias y tareas seleccionadas para cumplir el objetivo del Sprint.                  |
| Increment / Definition of Done | Resultado integrado que cumple criterios de aceptación, pruebas, revisión y condiciones de calidad. |

Una historia estará preparada para un Sprint cuando sea comprensible, tenga criterios de aceptación, dependencias identificadas y datos de prueba. Estará terminada cuando cumpla los

criterios, pase las pruebas y CI, no incluya secretos, tenga migraciones o documentación necesarias y haya recibido revisión humana mediante Pull Request.

## 5. Product Backlog inicial y trazabilidad

| ID    | Historia                            | Prioridad | Pts. | Sprint |
|-------|-------------------------------------|-----------|------|--------|
| HU-01 | Inicio de sesión seguro             | Alta      | 3    | 1      |
| HU-02 | Lista y búsqueda de pacientes       | Alta      | 5    | 1      |
| HU-03 | Crear y editar paciente             | Alta      | 5    | 1      |
| HU-04 | Expediente del paciente             | Alta      | 5    | 1      |
| HU-05 | Acceso rápido a WhatsApp            | Media     | 2    | 2      |
| HU-06 | Registrar consulta                  | Alta      | 5    | 2      |
| HU-07 | Historial de consultas              | Alta      | 3    | 2      |
| HU-08 | Registrar mediciones                | Alta      | 3    | 2      |
| HU-09 | Comparar evolución                  | Alta      | 5    | 2      |
| HU-10 | Notas, preferencias y restricciones | Alta      | 3    | 2      |
| HU-11 | Formalizar notas con IA             | Media     | 8    | 3      |
| HU-12 | Resumen de seguimiento              | Media     | 5    | 3      |
| HU-13 | Documentos esenciales               | Media     | 5    | 3      |
| HU-14 | Auditoría y manejo de errores       | Media     | 5    | 3      |

Los puntos son una estimación inicial y deberán revisarse durante el refinamiento. Si la capacidad real demuestra que el Sprint 2 o 3 está sobrecargado, el Product Owner reordenará historias de prioridad media antes de comprometer la fecha de entrega.

## 6. Plan de Sprints hasta el 10 de octubre

El calendario toma como inicio operativo el 21 de septiembre de 2026 y como fecha límite el 10 de octubre. El Agente Orquestador comienza desde el Sprint 1 porque su propósito es acompañar y coordinar el desarrollo distribuido, no incorporarse únicamente al final.

| <b>Sprint / fechas</b>     | <b>Sprint Goal</b>                                         | <b>Historias y trabajo principal</b>                                                                                                          | <b>Resultado</b>                                                                                       |
|----------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Sprint 1<br>21–26 sep.     | Establecer la base operativa y el contexto compartido.     | HU-01 a HU-04; repositorio; React/Django/Postgre SQL; CI inicial; documentación base; configuración inicial del contexto del Orquestador.     | Acceso y gestión básica de pacientes; repositorio y orquestador preparados para acompañar el proyecto. |
| Sprint 2<br>27 sep.–3 oct. | Completar el seguimiento nutricional e integrar las capas. | HU-05 a HU-10; consultas, mediciones, historial, evolución; uso activo del Orquestador para contratos React–Django–datos y revisión de ramas. | Flujo principal de seguimiento demostrable e integrado.                                                |
| Sprint 3<br>4–9 oct.       | Completar funciones secundarias, IA y estabilizar.         | HU-11 a HU-14; IA de notas; resumen; documentos; auditoría; pruebas E2E; seguridad; consolidación del Orquestador; documentación y demo.      | MVP probado y preparado para entrega.                                                                  |
| Entrega<br>10 oct.         | Validar y presentar.                                       | Verificación final, evidencias, manual, documento,                                                                                            | Entrega académica final.                                                                               |

|  |  |                             |  |
|--|--|-----------------------------|--|
|  |  | repositorio y demostración. |  |
|--|--|-----------------------------|--|

La fecha es fija; por ello, el alcance es la variable de ajuste. Si existe retraso, se conservarán primero autenticación, pacientes, consultas, mediciones, historial, comparación de evolución, integración React–Django, persistencia y pruebas críticas.

## 7. Arquitectura y tecnologías

GestNutri utilizará una arquitectura cliente-servidor. React implementará la interfaz; Django y Django REST Framework expondrán la API y concentrarán autenticación, autorización, validaciones y lógica de negocio; PostgreSQL almacenará la información. Las funciones de IA del producto se invocarán desde el backend para evitar exponer credenciales en el navegador.

| Capa                       | Tecnología / responsabilidad                                           |
|----------------------------|------------------------------------------------------------------------|
| Frontend                   | React, JavaScript/JSX, HTML, CSS y Vite.                               |
| Backend                    | Python, Django y Django REST Framework.                                |
| Datos                      | PostgreSQL y migraciones de Django.                                    |
| Versionado                 | Git y GitHub.                                                          |
| CI ( Integración continua) | GitHub Actions.                                                        |
| IA de desarrollo           | Asistencia individual en cada computadora + Agente Orquestador global. |
| IA del producto            | Formalización controlada de notas mediante backend.                    |

Flujo arquitectónico: Usuario → React → API REST Django/DRF → PostgreSQL.  
 Cuando una función requiera IA: React → Django → servicio de IA → propuesta → Django → React → revisión del nutricionista. Los cálculos de evolución serán determinísticos y verificables.

## 8. Desarrollo distribuido con Git y GitHub

Cada integrante tendrá un clon local del mismo repositorio. main contendrá la versión estable; develop será la rama de integración; y las historias se desarrollarán en ramas feature/HU-XX-descripcion. También podrán utilizarse fix y docs.

| Paso           | Procedimiento                                                                |
|----------------|------------------------------------------------------------------------------|
| 1. Sincronizar | Actualizar develop y revisar cambios recientes.                              |
| 2. Crear rama  | Crear o actualizar la rama de la historia.                                   |
| 3. Desarrollar | Programar y probar localmente, con IA de apoyo cuando resulte útil.          |
| 4. Publicar    | Commit descriptivo y push a GitHub.                                          |
| 5. Revisar     | Pull Request a develop, CI, revisión humana y observaciones del Orquestador. |
| 6. Integrar    | Corregir, aprobar y fusionar; el resto del equipo vuelve a sincronizar.      |

GitHub será la fuente oficial del código y del estado compartido. El equipo no integrará el sistema enviando carpetas completas entre computadoras. Los commits utilizarán prefijos como feat, fix, test, docs, refactor y chore.

## 9. Forma de trabajo con inteligencia artificial

Cada integrante podrá utilizar IA desde su computadora sobre la tarea y rama que tenga asignadas. Esta asistencia local puede ayudar a comprender código, proponer implementaciones, crear pruebas, revisar errores o redactar documentación. No constituye un proyecto independiente y debe respetar las decisiones comunes almacenadas en el repositorio.

| Área              | Uso de IA local                                              | Validación humana                                             |
|-------------------|--------------------------------------------------------------|---------------------------------------------------------------|
| Frontend / Jilder | Componentes, formularios, estados, consumo de API y pruebas. | Verificar experiencia, contrato de API y funcionamiento real. |
| Backend / Genny   | Endpoints, serializers, servicios, permisos y pruebas.       | Verificar reglas, seguridad y compatibilidad.                 |

|               |                                                 |                                              |
|---------------|-------------------------------------------------|----------------------------------------------|
| Datos / Diana | Modelos, relaciones, migraciones y consultas.   | Verificar integridad y efectos sobre API.    |
| QA / Luisa    | Casos de prueba, CI, revisión y automatización. | Confirmar cobertura crítica y confiabilidad. |

Antes de aceptar una sugerencia, el autor debe comprenderla, revisar que respete la historia y arquitectura, ejecutar pruebas y confirmar que no modifique archivos fuera del alcance. La IA no recibirá secretos ni datos reales de pacientes. Cuando una tarea dependa de otra área, el equipo acordará primero el contrato compartido; la IA no deberá inventarlo.

## 10. Agente Orquestador con contexto global

El Agente Orquestador es distinto de la IA local de cada desarrollador. Su propósito es mantener una visión integral de GestNutri y ayudar a coordinar el trabajo que se publica desde las cuatro computadoras. GitHub conserva el código y el historial; el orquestador interpreta ese estado, relaciona cambios y formula recomendaciones.

| Tipo de contexto    | Contenido                                                                                           |
|---------------------|-----------------------------------------------------------------------------------------------------|
| Contexto base       | Alcance, Product Goal, arquitectura, tecnologías, convenciones, seguridad y Definition of Done.     |
| Contexto del Sprint | Sprint Goal, historias seleccionadas, criterios de aceptación, tareas y dependencias.               |
| Contexto dinámico   | Ramas, commits, Pull Requests, cambios de API, migraciones, pruebas, Issues y decisiones recientes. |

Flujo operativo: cada desarrollador trabaja localmente con IA → publica su rama en GitHub → el Orquestador analiza repositorio, documentación, historias y Pull Requests → identifica dependencias, inconsistencias o impactos → emite recomendaciones → el responsable corrige y prueba → CI y revisión humana → Pull Request aprobado → integración en develop → actualización del contexto global.

El orquestador complementará a Git detectando conflictos semánticos. Por ejemplo, si React espera `phoneNumber` y Django devuelve `phone`, Git puede fusionar ambos cambios sin conflicto textual, pero la aplicación fallará. El orquestador deberá advertir la incompatibilidad. También podrá detectar migraciones faltantes, pruebas no actualizadas, duplicación de trabajo o documentación obsoleta.

El agente puede sugerir ideas porque conoce el proyecto completo, pero no puede convertirlas por sí mismo en requisitos. Las propuestas de alcance deben pasar por el Product Owner; las decisiones de proceso corresponden al Scrum Team; y las decisiones técnicas importantes requieren revisión humana. El orquestador no aprobará su propio trabajo, no fusionará directamente a main, no eliminará cambios ajenos y no sustituirá al Scrum Master.

## 11. Gestión de requisitos y cambios

Los requisitos se administrarán como historias en el Product Backlog. Una solicitud nueva se registrará, se analizará por valor, esfuerzo, dependencias, riesgo e impacto sobre la fecha y será ordenada por el Product Owner. Los cambios que afecten arquitectura, contratos de API, datos o seguridad deberán documentarse antes de integrarse.

| Estado del cambio                     | Acción                                                                                    |
|---------------------------------------|-------------------------------------------------------------------------------------------|
| Cambio pequeño dentro de una historia | Se aclara en criterios y se implementa dentro del Sprint si no compromete el Sprint Goal. |
| Nueva necesidad                       | Se crea una historia o Issue y vuelve al Product Backlog.                                 |
| Cambio de arquitectura/API/datos      | Se documenta la decisión y se comunica a los integrantes y al Orquestador.                |
| Cambio que amenaza la fecha           | El Product Owner reordena alcance; no se introduce silenciosamente en el Sprint.          |

## 12. Gestión de calidad, pruebas y CI

La calidad se verificará durante cada Sprint. Ninguna historia podrá considerarse Done si incumple sus criterios, mantiene pruebas críticas fallidas o no supera las revisiones acordadas.

| Nivel         | Qué verifica                                     | Responsable principal | Momento                          |
|---------------|--------------------------------------------------|-----------------------|----------------------------------|
| Unitarias     | Reglas, cálculos y funciones aisladas.           | Autor del cambio      | Durante desarrollo.              |
| API / backend | Serializers, permisos, endpoints y errores.      | Backend / Datos       | Antes del PR.                    |
| Frontend      | Componentes, estados y validaciones.             | Frontend              | Antes del PR.                    |
| Integración   | Compatibilidad React-API-datos.                  | Equipo                | PR / cierre de historia.         |
| E2E           | Flujos principales del usuario.                  | QA + equipo           | Sprint 3 y regresiones críticas. |
| IA            | Que la formalización no agregue hechos ausentes. | QA + autor            | Sprint 3.                        |

HU-09 deberá probar aumento, disminución, igualdad, ausencia de consulta previa y datos nulos o inválidos. GitHub Actions ejecutará verificaciones automáticas en Pull Requests. Un CI exitoso será requisito de integración, pero no reemplazará la revisión humana.

### 13. Seguridad, riesgos y comunicación

Django controlará permisos y validaciones del servidor. Contraseñas y secretos se manejarán con los mecanismos seguros del framework y variables de entorno; no se almacenarán claves en Git. En desarrollo se utilizarán datos ficticios o anonimizados. La IA del producto deberá ser degradable: si el servicio no está disponible, las notas manuales seguirán funcionando.

| Riesgo                          | Prob. | Impacto | Responsable            | Respuesta                                             |
|---------------------------------|-------|---------|------------------------|-------------------------------------------------------|
| Retraso antes del 10 de octubre | Alta  | Alto    | Product Owner + equipo | Proteger núcleo del MVP y reordenar funciones medias. |
| Conflictos entre ramas          | Media | Alto    | Developers             | Ramas pequeñas, sincronización,                       |

|                                   |       |       |                    |                                                        |
|-----------------------------------|-------|-------|--------------------|--------------------------------------------------------|
|                                   |       |       |                    | PR y Orquestador.                                      |
| Código incorrecto generado con IA | Media | Alto  | Autor del cambio   | Comprensión, pruebas, revisión cruzada y CI.           |
| Incompatibilidad React–Django     | Media | Alto  | Frontend + Backend | Contrato de API y revisión semántica.                  |
| Pérdida de contexto               | Media | Medio | Scrum Team         | Documentación versionada y GitHub como fuente oficial. |
| Fuga de información a IA          | Baja  | Alto  | Todo el equipo     | Datos ficticios, minimización y no compartir secretos. |
| Falla del servicio de IA          | Media | Medio | Backend / QA       | Mantener flujo manual disponible.                      |

La Daily Scrum será el principal punto de coordinación operativa. Issues y Pull Requests registrarán trabajo y decisiones técnicas. El Scrum Master dará seguimiento a impedimentos y al uso correcto de Scrum; el Product Owner mantendrá el orden del Product Backlog; y los Developers adaptarán diariamente el plan para alcanzar el Sprint Goal.

#### 14. Seguimiento y métricas

| Indicador                           | Uso                                                                  |
|-------------------------------------|----------------------------------------------------------------------|
| Historias terminadas / planificadas | Observar cumplimiento del Sprint sin medir productividad individual. |
| % de historias que cumplen DoD      | Controlar calidad real del Increment.                                |
| Pull Requests pendientes            | Detectar cuellos de botella de revisión e integración.               |
| Pruebas o CI fallidos               | Identificar inestabilidad técnica.                                   |
| Defectos abiertos críticos          | Decidir si el Increment está listo para demostración o entrega.      |

Las métricas se revisarán durante el Sprint y especialmente en Review y Retrospective. La velocidad, si se utiliza, servirá únicamente como referencia de planificación del equipo y no como evaluación individual.

## **15. Entregables y criterios de aceptación**

Los entregables serán: repositorio GitHub con historial; frontend React; backend Django/DRF; modelo y migraciones PostgreSQL; Product Backlog y evidencia de Sprints; pruebas y CI; documentación de API; reglas y descripción del Agente Orquestador; guía de instalación; manual breve; plan de gestión; MVP demostrable y presentación final.

El MVP será aceptado cuando un usuario autorizado pueda iniciar sesión, administrar pacientes, registrar consultas y mediciones, consultar historial, visualizar correctamente la evolución, guardar notas sin depender de IA y utilizar opcionalmente la formalización con revisión humana. También deberá demostrarse que React consume la API Django, los datos persisten correctamente y el trabajo distribuido fue integrado mediante GitHub, Pull Requests, CI y revisión.

## **16. Conclusiones**

El plan organiza GestNutri alrededor de una fecha fija, un único producto y un equipo distribuido. Scrum proporciona objetivos, inspección y adaptación; Git y GitHub permiten integrar el trabajo de cuatro computadoras; la IA local apoya a cada desarrollador; y el Agente Orquestador aporta contexto global para detectar dependencias e inconsistencias desde el primer Sprint. La combinación de Product Goal, backlog trazable, pruebas, CI, gestión de riesgos y revisión humana permite mantener el proyecto controlado sin convertir el plan en documentación innecesariamente extensa.

## Referencias

- Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P., & Roberts, K. (2024). Artificial intelligence risk management framework: Generative artificial intelligence profile (NIST AI 600-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.600-1>
- Django REST Framework. (2026). Quickstart. <https://www.django-rest-framework.org/tutorial/quickstart/>
- Django Software Foundation. (2026). Django documentation (Version 5.2). <https://docs.djangoproject.com/en/5.2/>
- GitHub. (2026). Pull requests documentation. GitHub Docs. <https://docs.github.com/en/pull-requests>
- React Team. (2026). Quick start. React. <https://react.dev/learn>
- Schwaber, K., & Sutherland, J. (2020). The Scrum guide. <https://scrumguides.org/scrum-guide.html>